FitNFree

Project Documentation

Last updated: September 28, 2026

Maintained by LixTeam (Team K)

Status: iOS 1.0 in App Store review (build 14); Android in closed testing

1. What FitNFree Is

FitNFree is an AI-assisted fitness, nutrition, sleep and wellness companion app. The product pitch: one app that adapts workouts, nutrition, sleep and recovery guidance to how the user is actually doing, instead of requiring four separate apps.

- Original name: FitFlex, renamed to FitNFree on September 6, 2026.
- Bundle ID / package name: com.teamk.fitnfree (same on iOS and Android).
- Platforms: iOS, Android, and a web app — all three from one codebase.
- Business model: freemium — free tier with limited daily AI actions; Premium subscription ($8.99/month) for unlimited AI and extra features.

2. Technology Stack

Client (the app)

Layer	Technology	Version
Framework	React Native via Expo (managed workflow)	Expo SDK 57, React Native 0.86.2
Language	TypeScript	~6.0.3
UI library	React	19.2.3
Navigation	Hand-rolled screen switch in App.tsx (no navigation library)	—
Styling	React Native StyleSheet, no CSS framework	—
State	Local component state + a few small hooks (no Redux/MobX)	—
Web target	react-native-web, static export via "expo export -p web"	^0.21.2
Backend SDK	@supabase/supabase-js	^2.112.1

Key Expo/RN packages in use: expo-camera, expo-image-picker (photo/food/equipment scans), expo-audio (ambient sound, breathwork, voice notes), expo-speech + expo-speech-recognition (voice input/output for the AI Coach), expo-location (outdoor activity tracking), expo-notifications (local notifications only — no push/remote notifications are used), expo-video, expo-iap (Apple/Google in-app purchases), react-native-maps, react-native-webview (YouTube embeds), react-native-svg.

Backend

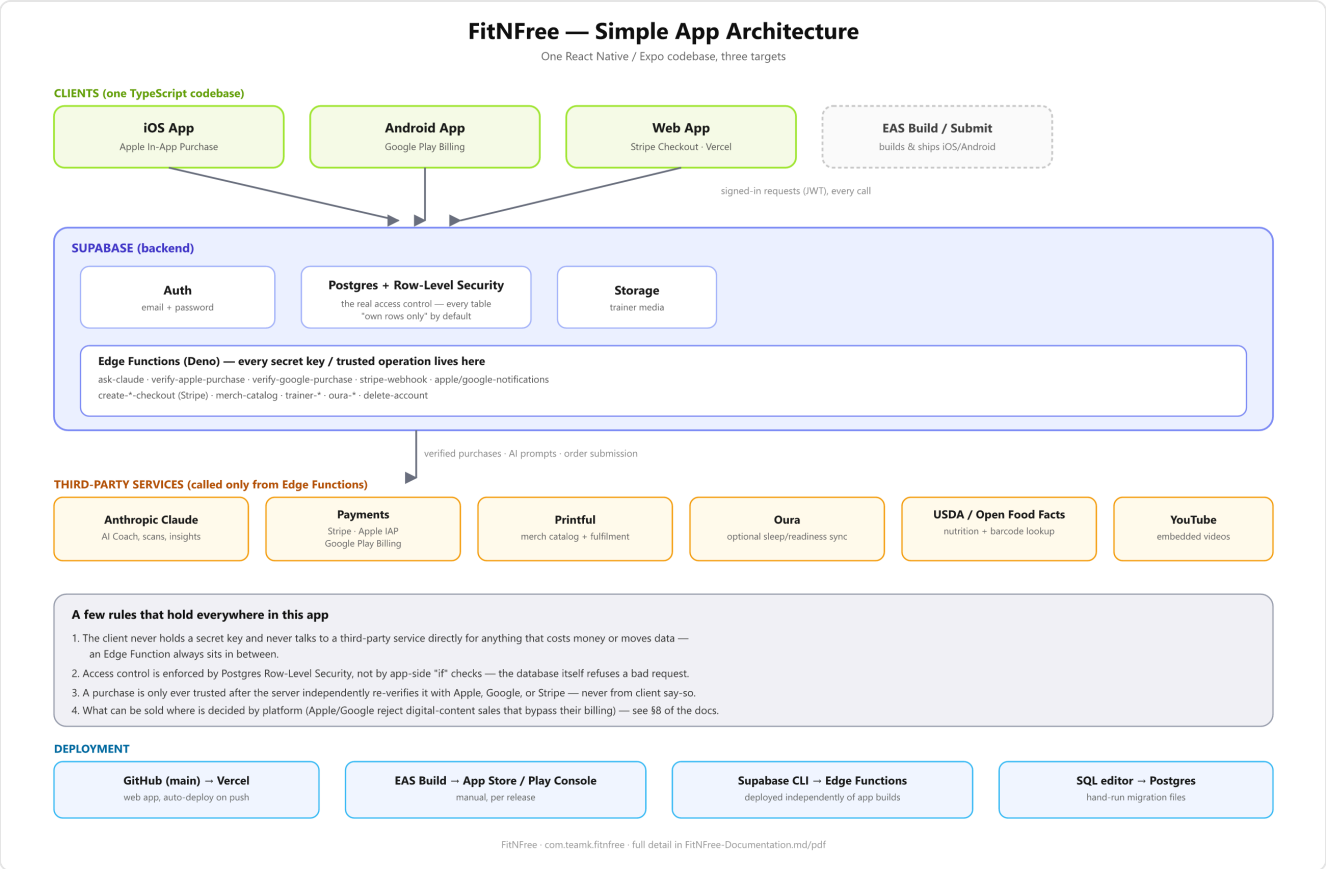
Piece	Technology
Database	Postgres, via Supabase
Auth	Supabase Auth (email + password)
Server logic	Supabase Edge Functions (Deno/TypeScript), one function per job
File storage	Supabase Storage (trainer media uploads)
Row-level security	Postgres RLS on every table — see Section 6

Third-party services

Service	What it's used for
Anthropic (Claude)	All AI features: AI Coach, equipment/food/menu scans, nutrition search, insights, form guardrails. Proxied through one Edge Function, ask-claude, so the API key never reaches the client.
Apple In-App Purchase (via expo-iap)	Premium subscription on iOS.
Google Play Billing (via expo-iap)	Premium subscription on Android.
Stripe	Premium subscription on web; physical merch; live trainer sessions; trainer payouts via Stripe Connect.
Printful	Print-on-demand merchandise fulfilment (order submission on payment).
Oura	Optional sleep/readiness data sync, only if the user connects their ring.
USDA FoodData Central + Open Food Facts	Nutrition search and barcode lookups.
YouTube	Embedded exercise/tutorial videos (own player component, native WebView / web iframe).
Vercel	Hosts the web build (static export), auto-deploys on push to main.
EAS (Expo Application Services)	Builds and submits the iOS/Android native binaries.
GitHub	Source control (DeLix-K/FitNFree, and the separate DeLix-K/LixTeam-Site).

3. Architecture

The diagram below is the simple architecture view. Full detail follows it.



Core principles

1. One codebase, three targets. The same React Native/Expo source compiles to an iOS app, an Android app, and a static web bundle. Platform differences are handled inline with Platform.OS checks (e.g. billing provider, a few layout tweaks) rather than separate codebases.
2. The client never talks to Postgres directly except through Supabase's REST/RLS layer, and never holds secret API keys. Every request carries the signed-in user's JWT, and Postgres Row-Level Security — not app code — is what actually decides what each user can read or write.
3. Anything that needs a secret key or must be trusted (AI calls, payment verification, admin-only writes) goes through an Edge Function, never straight from the client to a third party.
4. Payment verification is always server-side. The client only ever hands the server an opaque transaction ID or purchase token; the corresponding Edge Function re-verifies it directly with Apple/Google/Stripe before granting anything.

Request flow examples

A normal data read/write (e.g. logging a workout):

```
App screen → supabase-js → Postgres (RLS enforces "own rows only") → response
```

An AI feature (e.g. asking the Coach a question):

```
App screen → askClaude()/askClaudeChat() (lib/claude.ts) → ask-claude Edge Function (checks the caller's session, enforces the free daily AI limit, optionally checks a shared cache, calls Anthropic) → reply → also saved to ai_history
```

A Premium purchase on iOS:

```
ProfileScreen/CoachScreen → startCheckout() (lib/billing.ts) → purchasePremiumIOS() (lib/iap.ts) → expo-iap / StoreKit → transaction ID → verify-apple-purchase Edge Function (re-checks the transaction with Apple's server) → profiles.is_premium = true
```

4. Project Structure

```
FitFlex/                    (repo root; folder name predates the FitNFree rename)
├── App.tsx                 Root component: auth gate, tab switching
├── index.tsx               Expo entry point, wraps App in SafeAreaProvider
├── app.json                Expo config: bundle IDs, permissions, plugins
├── eas.json                EAS Build/Submit profiles
├── screens/                One file per top-level screen (35 files)
├── components/             Reusable UI + feature components (80 files)
├── hooks/                  Small shared hooks (e.g. useAiGate)
├── lib/                    Client-side data/business logic, one file per domain
│                            (courses.ts, nutrition.ts, trainers.ts, iap.ts, ...)
├── plugins/                Custom Expo config plugins (e.g. disabling a push entitlement)
├── assets/                 Icons, images, store screenshots, seed photos
├── supabase/
│   ├── functions/          One folder per Edge Function (Deno)
│   ├── schema.sql          Original base schema
│   └── *.sql               ~56 incremental migration files, one per feature/fix,
│                             run by hand in the Supabase SQL editor (no migration
│                             runner/ORM — see Section 7)
├── docs/                   Store listing copy, screenshots, this documentation
└── scripts/                One-off maintenance scripts
```

There is no src/ folder — screens/components/lib sit at the repo root. There is no ORM: all database access goes through the supabase-js query builder or raw SQL in Edge Functions.

5. Data Model (Postgres Tables)

Grouped by feature area. Every table has Row-Level Security enabled (see Section 6).

Area	Tables
Accounts	profiles (extends auth.users: is_premium, premium_source, is_admin, Apple/Google/Stripe purchase links)
Workouts	exercises, workout_plans, workout_plan_exercises, workout_logs, exercise_set_logs, saved_exercises, programs, program_plans, plan_schedule
Nutrition	meal_logs, hydration_logs
Sleep	sleep_logs, sleep_behavior_tags, oura_connections
Wellness / habits	mood_logs, habits, habit_logs, habit_freeze_uses
Streaks & challenges	streak_freezes, streak_freeze_uses, challenges, challenge_participants, challenge_stages, challenge_teams, challenge_team_members, challenge_activity, challenge_reactions, challenge_invites

Area	Tables
AI	ai_history (every AI reply, per user), ai_response_cache (shared cache for routine/cacheable AI calls — see Section 9)
Trainers	trainer_profiles, trainer_orders, trainer_time_slots, trainer_session_packages, trainer_session_credits, trainer_reviews, trainer_form_reviews, trainer_messages, blocked_users, message_reports
Courses & guides	courses, course_lessons, course_enrollments, course_lesson_progress, digital_products, digital_product_content, digital_product_purchases
Merch	merch_orders (product catalog itself is not stored — see Section 9)
Outdoor	outdoor_activities

6. Authentication, Authorization & Security

- Auth: Supabase Auth, email + password. profiles extends auth.users 1:1.
- Row-Level Security is the real access control, not app-side checks. Almost every table's default policy is "a user can only read/write their own rows" (auth.uid() = user_id). Public catalogs (exercises, courses, trainer profiles) are readable by anyone but writable only by the admin.
- Admin access: profiles.is_admin, a boolean set once by direct SQL, checked via auth.uid(). This replaced an earlier pattern that checked the user's email from their JWT — fixed in security_hardening.sql because that depended on Supabase Auth settings staying correctly configured forever, unlike every other privileged flag in the app.
- Never trust the client for money or entitlements. is_premium, purchase links, and order status are only ever written by Edge Functions using a service-role client, after independently verifying the purchase/payment with Apple, Google, or Stripe.

> *A recurring gotcha worth knowing: service_role needs its own explicit Postgres grant per table, even for tables that predate the feature needing service-role access. This has silently blocked a new Edge Function's reads/writes several times across the project's history (trainer marketplace, courses, digital products). When adding a new Edge Function that touches a table via the service role, check that a "grant ... to service_role" exists for every table it reads or writes.*

7. Database Migrations

There is no migration framework (no Prisma/Drizzle/Supabase CLI migrations directory in active use). Each supabase/*.sql file is a hand-written, incremental migration — one file per feature or fix, applied by pasting into the Supabase SQL editor (or via "supabase db query --file ... --linked"). Files use "create table if not exists" and "alter table ... add column if not exists", so they're safe to re-run.

> *When adding a new migration: name it after the feature (e.g. trainer_listed_for_chat.sql), keep it additive/backwards-compatible where possible (existing app builds in the stores must keep working against the new schema), and if a new policy references another table via a subquery, create that table earlier in the same script — Postgres has no forward declaration, and a mid-script failure leaves the rest of the script un-run.*

8. Payments — Per Platform

Apple and Google require digital content/features unlocked inside their apps to be sold through their own billing systems. FitNFree's billing is therefore split by platform:

What	iOS	Android	Web
Premium subscription	Apple In-App Purchase	Google Play Billing	Stripe Checkout
Physical merch	Stripe (exempt)	Stripe (exempt)	Stripe
Live trainer sessions	Stripe (exempt)	Stripe (exempt)	Stripe
Courses / guides / custom plans	Hidden	Hidden	Stripe (only place these sell today)

lib/storeRules.ts (CAN_SELL_DIGITAL_CONTENT) is the single flag that gates this — true only on web.

- lib/iap.ts wraps expo-iap for both stores: purchasePremiumIOS(), purchasePremiumAndroid(), restorePremiumPurchases(), and verifyOwnedPremium() (handles the case where the store account already owns the subscription).
- lib/billing.ts (startCheckout()) is the single entry point every "Upgrade to Premium" button calls; it picks the right path by Platform.OS and de-dupes concurrent taps.
- Server-side verification: verify-apple-purchase and verify-google-purchase Edge Functions independently re-check the transaction/purchase token with Apple/Google before setting profiles.is_premium. Each provider can only revoke premium it granted (profiles.premium_source tracks who granted it: ‘stripe’, ‘apple’, or ‘google’).
- Webhooks: stripe-webhook, apple-notifications, google-notifications keep subscription status in sync after the initial purchase (renewals, cancellations, refunds).

9. AI Integration

All AI calls go through one Edge Function, ask-claude, so the Anthropic API key never reaches the client and the freemium limit is enforced server-side (not just in the UI — calling the function directly can't bypass it).

- Free tier: 5 AI actions/day, counted from ai_history rows created that day. Premium: unlimited.
- Two client entry points (lib/claude.ts): askClaude(prompt, image?, opts?) for one-shot calls (scans, lookups, cues), and askClaudeChat(messages, system?) for the conversational AI Coach.
- Cost controls (added September 2026): a one-shot call can be marked routine: true for genuinely routine, non-personalized lookups (nutrition search, per-exercise cues/guardrails, equipment/food/menu photo identification). For those calls, ask-claude checks a shared cache (ai_response_cache, keyed by a hash of model+prompt+system+image) before calling Anthropic at all,

and falls back to claude-haiku-4-5-20251001 instead of claude-sonnet-5 on a miss. The AI Coach chat, coach memory, mood reflection, daily briefing, insights, sleep insight, and bedtime stories are never marked routine — they always get the full model and are never cached, since they're personalized or meant to feel premium.

- Merch catalog is the one place the client calls a third party directly (via an Edge Function proxy, merch-catalog) rather than caching data in Supabase — Printful stays the single source of truth for products, fetched live on every screen load.

10. Build & Deployment

Target	Tool	Trigger
iOS / Android native builds	EAS Build (eas.json production profile)	Manual: eas build --profile production --platform ios|android
iOS / Android store submission	EAS Submit (App Store Connect API key; Google service-account JSON)	Manual, often --auto-submit on the build command
Web	Vercel, static export (expo export -p web)	Automatic on every push to main
Edge Functions	Supabase CLI (supabase functions deploy <name>)	Manual, per function, independent of app builds
Database migrations	supabase db query --file <path> --linked	Manual, per migration file

> *Because of the Vercel auto-deploy on main, in-progress work happens on feature branches and only merges to main when it's ready to go live on the web (or, for native-only changes, when it's safe to bundle into the next store build). main is kept in sync with whatever is actually live/submitted, not with work in progress.*

Environment variables (client-side, EXPO_PUBLIC_*, set via "eas env:set" for cloud builds — EAS Build does not read local .env files): Supabase URL and anon key, USDA API key, Oura client ID. Server-side secrets (Anthropic key, Stripe key, Apple/Google credentials) are Supabase Edge Function secrets, never shipped in the app.

11. Feature Map

Feature	Where	Notes
Workouts	PlansScreen, ProgramDetailScreen, StreaksScreen	home/gym/outdoor plans, streaks, badges
Equipment camera scan	EquipmentScanScreen → PhotoScanScreen	Also offers a Premium-gated "how to use it" tutorial video (lib/equipmentVideos.ts)
Nutrition	NutritionScreen, LogMealModal	Search, photo/barcode/voice logging; USDA + Open Food Facts backed
AI Coach	CoachScreen	Conversational, 3 personality options, voice in/out
Sleep & recovery	SleepScreen	Manual entry or Oura auto-sync; breathwork, ambient sound, AI bedtime stories
Wellness & habits	WellnessHubScreen, HabitsScreen	Mood check-ins, forgiving streak-freeze habit tracker

Feature	Where	Notes
Squad challenges	ChallengesScreen	Leaderboards, real-time activity feed with reactions
Trainer marketplace	TrainersScreen, TrainerProfileScreen, TrainerDashboardScreen	Chat, live sessions, report/block; needs verified Stripe payouts to sell, can be listed chat-only by an admin flag
Content library	CoursesScreen, DigitalProductsScreen	5 categories; free starters + Premium unlocks the rest; scheduled releases
Merch store	MerchScreen	Live Printful catalog, Stripe checkout, auto-submits to Printful on payment
Premium subscription	Cross-cutting	See Section 8

12. Known Gotchas

Worth reading before touching the same area again.

1. service_role grants — see Section 6. Recurs every time a new Edge Function touches an existing table via the service role.
2. EAS Build doesn't read local .env files. A missing EXPO_PUBLIC_* env var in EAS's own store causes a silent, blank frozen splash screen in the built app (no error UI in a release build) — always check the build log for the "Environment variables ... loaded" line.
3. react-native-health-style old-architecture native modules don't work on this RN version — the project is forced onto the new architecture (RN ≥ 0.82). Any future native module needs explicit new-architecture / Nitro / TurboModule support (this bit the original choice of react-native-iap, which was replaced with expo-iap).
4. A first-ever Apple subscription must be submitted together with an app version — App Store Connect will reject subscriptionSubmissions with FIRST_SUBSCRIPTION_MUST_BE_SUBMITTED_ON_VERSION otherwise. Use the "Add for Review" flow on the subscription group and the version together, not the API in isolation.
5. Apple Guideline 3.1.2 — an app offering auto-renewable subscriptions needs a functional Terms of Use (EULA) link in the App Store Description metadata (not just inside the app), unless a custom EULA is set in App Store Connect.
6. Google Play "License testing" only makes a Google account's purchases free/test — being on the closed-testing Testers email list is a separate, different list and does not grant that. A tester also generally needs to visit the app's Play Store testing opt-in link, not just be listed.
7. npm lockfile drift: this machine's local npm has produced lockfiles EAS's "npm ci" rejected as inconsistent. Regenerate with "npx npm@10 install" (not the locally installed npm) if a build fails with "Missing: X from lock file".

13. Store Presence (Snapshot)

- iOS: App Store Connect app id 6809598004. Version 1.0, build 14, submitted for review 2026-09-28 alongside the Premium Monthly subscription (com.teamk.fitnfree.premium.monthly). Two earlier rejections in this round: Guideline 2.1 (demo account credentials), 2.5.4 (an unused background-audio capability), 2.1(b) (subscription not submitted with the version), and 3.1.2 (missing EULA link) — all addressed.
- Android: Google Play Console, package com.teamk.fitnfree, closed testing ("Alpha") track, version code 6 live. Production track not yet used.
- Web: live at fitflex-lilac.vercel.app and fitnfree-lilac.vercel.app (same deployment, two domains). A real custom domain has not yet been purchased.
- Company/legal site: a separate project, LixTeam-Site (github.com/DeLix-K/LixTeam-Site), a small static Astro site — one page per app, each with its own Privacy Policy / Terms / Support / Delete-account pages. Built but not yet the live source of truth for the store listings (those still point at an older GitHub Pages site).

14. In Progress / Not Yet Merged

As of this document's date, four feature branches exist that are ready but intentionally not yet merged into main (to avoid touching anything while the iOS submission above is mid-review):

- fix/app-review-round-2 — the build-14 fixes (background audio, sign-in whitespace).
- feature/content-library — the categorised courses/guides library described in Section 11.
- feature/equipment-tutorial-videos — the equipment scan tutorial video.
- feature/ai-cost-controls — the AI caching/model-tiering described in Section 9 (the server-side half is already deployed and live; only the client-side "mark this call routine" changes are waiting on this branch).

15. How to Update This Document

1. Edit the Markdown source first — FitNFree-Documentation.md, kept in the repo under docs/documentation/, alongside the code it describes.
2. Regenerate the PDF from it (ask Claude to do this) and update FitNFree-Architecture.svg/png if the architecture changed.
3. Commit both. This folder is not auto-updated — it only reflects reality as of the last time someone (or Claude) deliberately refreshed it.